
EDA Bench: An Execution-Based Benchmark for Language Model Agents on Functional PCB Construction

Dhruv Saini
Independent Researcher
dhruv9saini@gmail.com

Abstract

Language-model agents are increasingly able to operate electronic design automation tools: they can search datasheets, select components, edit KiCad projects, run design checks, and iterate after failures. Existing evaluation signals do not directly test the property that matters for PCB-producing agents: whether the realized board preserves the target design’s external electrical behavior. We introduce EDA Bench, an execution-based benchmark for KiCad PCB reconstruction. EDA Bench contains 35 released tasks derived from public open-hardware projects spanning connector breakouts, power-path circuits, microcontroller systems, motor drivers, camera and compute adapters, and high-density carriers. Each harness receives a prompt and a standard KiCad environment, then must produce a complete KiCad project. Submissions are graded by an oracle that parses the realized PCB, extracts routed copper, checks declared external ports, models trace parasitics and short faults, runs ngspice simulations, and applies KiCad ERC and DRC checks. The score is based on observable external-I/O behavior rather than text overlap, screenshot similarity, component inventory, or reference-designator matching. We validate the artifact with reference, fail, and mutation canaries: all 35 released frozen references score 1.0, and all 35 released structural-failure canaries score at most 0.15. Initial baselines show that the strongest evaluated web-enabled harness scores 4.58% overall, building 21 of 35 projects while failing most functional checks. These results show that EDA Bench measures a capability missed by weaker proxies: parseable or plausible PCB artifacts are not necessarily electrically functional artifacts.

1 Introduction

Language-model agents can now operate development tools over long horizons. In software, this has made execution-based evaluation a central benchmark design pattern: an agent modifies an artifact, runs tools, observes failures, and is evaluated by executable checks rather than by surface-form similarity. SWE-bench evaluates agents on real GitHub issues whose solutions must pass repository tests [Jimenez et al., 2023]. OSWorld extends execution-based evaluation to open-ended desktop and web tasks in real computer environments [Xie et al., 2024]. ProgramBench evaluates a still broader reconstruction setting in which agents rebuild software from an executable and documentation, with behavioral tests used to compare against the reference program [Yang et al., 2026]. These benchmarks share a common principle: the evaluated artifact should be judged by behavior in an execution environment.

Hardware design has an analogous workflow. Engineers search for parts, inspect datasheets, draft schematics, route boards, run ERC and DRC checks, simulate where possible, and produce manufacturing files. For electronic design automation, however, execution-based testing is substantially

harder. Correctness depends on routed connectivity, electrical behavior, manufacturability constraints, and external interfaces rather than only on executable program behavior. Existing CAD and EDA benchmarks cover important parts of this space, including text-to-CAD generation [Khan et al., 2024], geometric CAD validation [Epoch AI, 2026, CAD Arena, 2026, Barkley et al., 2026], schematic generation [Al Hasan et al., 2026, Zou et al., 2026, Luo et al., 2026], schematic understanding [Lu et al., 2026], board-level schematic design [Qiu et al., 2026], hardware bug repair [Cui et al., 2026], and PCB placement and routing reasoning [Li et al., 2026]. These evaluations motivate execution-based assessment for engineering artifacts, but they do not directly test whether an agent can reconstruct a routed KiCad PCB that preserves the target board’s external electrical behavior.

EDA Bench addresses this gap by evaluating complete agent harnesses on source-backed KiCad PCB reconstruction tasks. A harness receives a fixed task prompt and runtime assets, then must write a complete KiCad project in `/workspace/final_project`. KiCad provides an open-source EDA environment for schematic capture, integrated simulation, PCB layout, 3D rendering, and manufacturing export [KiCad Project, 2026]. The released tasks are based on public hardware projects, and the default evaluated harnesses are web-enabled. This creates a tradeoff between reproducibility and realism similar to public software-engineering benchmarks: public sources make the benchmark auditable, but web-enabled agents may benefit from locating and adapting upstream material.

The benchmark is built around a functional scoring principle: evaluate the observable behavior of the realized board at its external electrical interfaces. A submitted project is parsed, checked, simulated, and compared against task contracts. ngspice supplies the open-source circuit-simulation backend used by the oracle [ngspice contributors, 2026]. A board can score well with a different internal implementation if it exposes the required external behavior. It can also fail with a visually plausible layout if external nets are missing, shorted, swapped, isolated, or not driven by the required circuitry. Although this evaluation requires more engineering effort than syntax checks, visual matching, or component-inventory comparison, positive scores are more informative because the submitted board must satisfy observable electrical-interface checks.

Contributions. This paper makes five contributions.

- We introduce EDA Bench, a public benchmark of 35 source-backed KiCad PCB reconstruction tasks spanning connector breakouts, power-path circuits, controller boards, compute adapters, motor drivers, and high-density carrier boards.
- We define a reproducible harness protocol in which agents receive only task-visible runtime packs and must produce complete KiCad projects, while references, source snapshots, contracts, canaries, and oracle internals remain part of the grading pack.
- We develop an execution-based PCB scoring oracle that evaluates realized board behavior through external-I/O contracts, routed-copper extraction, ngspice simulation, task-family checks, active-circuit realization checks, KiCad ERC and DRC diagnostics, and manufacturability caps.
- We validate the scoring artifact with frozen references, structural fail canaries, mutation canaries, and invariance controls, showing that the oracle accepts human-designed references while rejecting common function-breaking PCB failures.
- We report public-provenance baselines for six web-enabled harness configurations, including final projects, grader outputs, usage summaries, cost estimates, and regrade commands, and show that evaluated frontier-model harnesses remain far from functional PCB reconstruction.

2 Benchmark overview

EDA Bench evaluates PCB reconstruction as an execution task. For each task, a harness receives a fixed prompt, runtime task pack, tool environment, model configuration, and access policy, then must produce a complete KiCad project in `/workspace/final_project`. The submitted project is graded automatically. The oracle parses the KiCad files, extracts external connectors and routed copper, applies task-declared stimuli and probes, runs simulation where applicable, checks ERC and DRC diagnostics, and assigns a normalized score based on the realized board behavior.

Table 1: EDA Bench evaluation pipeline. Each submission is evaluated as a complete harness output rather than as text, a screenshot, or a component list.

Stage	Artifact or operation
Task input	The harness receives a fixed prompt, runtime task pack, tool environment, model configuration, and access policy.
Harness execution	The agent operates in the Dockerized KiCad environment and writes a project to <code>/workspace/final_project</code> .
Project parsing	The grader locates KiCad files and extracts connectors, pads, nets, board outline, and routed copper geometry.
Contract binding	External ports from the submitted board are matched to task-declared I/O contracts and expected measurements.
Execution checks	The grader applies KiCad ERC and DRC diagnostics, ngspice simulations, routed-I/O checks, parasitic models, and short-fault checks.
Score reporting	The oracle returns a normalized task score, hard-cap diagnostics, build status, and provenance artifacts.

Table 2: Task distribution and representative categories in EDA Bench.

Difficulty	Tasks	Representative task families
Very easy	2	Compact connector breakouts and small adapters
Easy	5	I/O breakouts, logic-level conversion, USB/UART basics
Medium	4	Power-path boards, serial transceivers, small controller boards
Hard	12	Compute adapters, sensor/camera boards, multi-connector carriers
Very hard	7	Dense mixed-signal boards and larger interface carriers
Extreme	5	Large baseboards and high-density compute boards

Table 1 summarizes the execution and grading pipeline. The benchmark is source-backed. Each released task is derived from a public hardware project and has a frozen reference design, but the evaluated runtime pack does not mount the reference project, source snapshot, canaries, grader contracts, or oracle implementation. This split makes the artifact auditable while preserving a clean separation between agent-visible inputs and grader-only materials. Because the released tasks are public-source and the default baselines are web-enabled, EDA Bench measures PCB reconstruction under a declared public-source access policy rather than closed-book circuit invention.

Scores should be interpreted as properties of complete evaluated harnesses, not isolated model weights. A harness may include prompting, web search, source discovery, KiCad tool use, repair loops, and project-generation logic. This systems-level framing is intentional because practical EDA automation depends on whether an agent can operate the full workflow, not only on whether a model can emit plausible schematic or layout text in one shot.

The central scoring claim is functional rather than visual. A board can score well with a different internal implementation if it exposes the required external electrical behavior under the task contract. Conversely, a board can fail despite looking plausible if external nets are missing, isolated, shorted, swapped, unrouted, or not driven by the required active circuitry. The score is therefore not a fabrication certificate, regulatory review, thermal analysis, EMC validation, or vendor-accurate silicon simulation. It is an execution-based measure of declared external-I/O behavior under the released oracle.

3 Source-backed tasks

EDA Bench contains 35 source-backed tasks. Each task is anchored to a public upstream hardware project and a frozen reference design. The task set covers six difficulty levels, shown in Table 2.

The task count reflects the cost of producing score-supported hardware evaluation instances rather than a preference for small benchmarks. Each released task requires a source design, a frozen KiCad reference, runtime and grading pack separation, an external-I/O contract, canary submissions, license triage, and oracle support. We therefore prioritize tasks with executable functional checks over a larger collection of weakly specified or weakly checked prompts.

The task catalog includes compact breakouts and converters such as `usb_c_female_breakout`, `mcp23017_breakout`, and `rs485_transceiver_breakout`. It also includes power and control boards such as `bq24295_power_path_board`, `picopd`, and `robotont_driver_board`, compute and camera or display adapters such as `cm4_csi_adapter`, `m2_pcie_adapter`, and `ov9281_dual_camera_board`, and large carrier boards such as `cm4_baseboard`, `jetson_orin_baseboard`, and `zynq_som`.

Each task has two artifact forms. The runtime task pack is mounted for the evaluated agent and contains the prompt, task metadata, and runtime assets. It does not include the reference project, source snapshot, canaries, grader contracts, or oracle implementation. The full grading pack is public and contains the reference project, upstream snapshot, canary submissions, license notes, `io_contract.json`, `functional_contract.json`, and local tests. This split supports reproducibility while avoiding direct mounting of the answer into the evaluated container.

4 Harness protocol

A harness is evaluated as a complete system. It receives the task prompt and must produce a KiCad project in `/workspace/final_project`. Built-in harnesses run inside Docker with KiCad, ngspice, Python, Node, internet access, and the configured model interface. The evaluated harnesses are Codex CLI with GPT-5.5 web search at low, medium, high, and xhigh reasoning, Pi with Gemini 3.1 Pro Preview web high, and Pi with DeepSeek V4 Pro web high.

This protocol evaluates system architecture, not only a model weight. A one-shot generator, a tool-using agent, and a multi-agent harness can all target the same task interface. This distinction matters because EDA work is iterative: the model may need to create files, run KiCad tools, inspect failures, search for upstream sources, and repair invalid output.

Each reported run includes provenance: task prompts, command lines, usage summaries, grader outputs, final projects, and generated artifacts. This provenance allows the final projects to be regraded without rerunning model harnesses.

5 Scoring oracle

The scorer evaluates a submitted KiCad project through explicit I/O simulation. Unsupported, missing, or malformed submissions receive zero score. Supported tasks declare their external ports, stimuli, probes, simulator models, expected measurements, and tolerances. For each submission, the oracle performs seven steps. It locates and parses the submitted KiCad project, runs KiCad ERC and DRC diagnostics, extracts connectors, pads, nets, and routed copper geometry, binds submitted external ports to the task contract, constructs task-declared simulation probes and stimuli, computes task-family measurements and hard caps, and returns a normalized score with diagnostic artifacts.

The primary score is a normalized task score in $[0, 1]$. A benchmark aggregate is a per-task difficulty-weighted mean:

$$S = \frac{\sum_{i=1}^N w(d_i) s_i}{\sum_{i=1}^N w(d_i)}, \quad w(d_i) \in \{1.0, 1.5, 2.0, 3.0, 4.0, 5.0\}. \quad (1)$$

We also report the unweighted mean over tasks. Build success means that the submission produced a KiCad project that the grader could parse and evaluate. It is not equivalent to functional success.

The main score comes from three observable properties of the submitted PCB: external connector semantics, routed-copper behavior, and task-required active and manufacturing plausibility. The oracle parses the project, extracts PCB geometry, applies deterministic ngspice stimuli at declared external ports, samples boundary waveforms, and applies hard caps for failures such as missing routed copper, swapped external pins, shorts, missing active circuitry, unpowered active devices, invalid high-speed pair families, and gross fabrication implausibility. Exact reference designators, component inventory, footprint similarity, outline similarity, and route-shape similarity are diagnostics, not primary score signals.

Within each task, declared measurements produce normalized sub-scores for external-port mapping, routed continuity, simulation behavior, and task-family realization checks. These sub-scores are

Table 3: Representative task-family checks layered on the generic external-I/O oracle. These are calibrated bindings on measured oracle signals, not vendor-accurate silicon or channel models.

Task family	Declared behavior	Common hard caps
Connector breakouts and simple adapters	Connector pins and power/ground pads, continuity, shorts, routed coverage, pin/net semantic agreement	Swapped pins, split nets, isolated pads, power-ground shorts
Serial and level-shifting interfaces	USB/UART/RS-485/logic-side boundary ports, signal transfer families, active-device realization	Missing active path, weak external mapping, missing routed I/O
Power-path and charger boards	Input, battery/load, configuration, and status ports, power path presence, external safety	Missing active power role, shorts, missing component-function profile
Camera/display/compute carriers	High-density connector families and differential pairs, routed families, pair plausibility	Missing high-speed pair families, weak routing, shorts, split nets
Large mixed-signal baseboards	Multi-rail power, GPIO, debug, serial, high-speed, and peripheral interfaces	Incomplete connector semantics, missing active realization, shorts

combined according to the task contract to form a soft functional score. Hard caps then impose upper bounds for structural failures that make the board externally incorrect or physically implausible, such as shorts, missing routed I/O, swapped connector pins, or absent active circuitry. The final task score is the resulting capped functional score. This design allows partial credit for partially correct external behavior while preventing submissions with severe structural faults from receiving high scores through unrelated checks.

6 Artifact validation

Because EDA Bench uses a task-specific oracle, the release includes controls that calibrate score interpretation. The pass canary is the frozen reference project and must score 1.0 under the same oracle. The fail canary removes active internal circuitry when available. Passive boards fall back to external boundary removal. The fail canary must score below the release threshold, currently 0.75.

Oracle validity. We validate the oracle as a contract-level evaluator, not as a vendor-accurate electrical simulator. Its intended validity criterion is whether score changes track engineering-relevant changes to declared external-I/O behavior. Positive controls test that human-designed reference boards are realizable and score 1.0. Negative controls test that structural changes a PCB engineer would regard as function-breaking receive low scores. Invariance controls test that benign representation changes, such as reference-designator renaming or plausible route-width edits, do not dominate the score.

A full local sweep on May 6, 2026 verified all 35 released task canaries: every pass canary scored 1.0, and every fail canary scored at most 0.15. The generated-mutation sweep graded 162 mutation canaries across released task families. All tasks passed, with every reference scoring 1.0 and every structural fail canary scoring at most 0.15. Mutations covered missing board outlines, unrouted external nets, isolated external pads, power or signal pads tied to ground, and missing high-speed-like routes for high-speed carrier tasks.

These controls do not show that the oracle replaces expert board review or captures vendor-specific silicon behavior. They show that, for the benchmark’s declared external-I/O contracts, the score accepts human-designed reference boards, rejects common function-breaking PCB mutations, and is invariant to several superficial implementation details.

Table 4: Oracle-validity evidence. The controls calibrate the released score as an external-I/O contract evaluator rather than as a complete manufacturing or vendor-silicon review.

Evidence	What it supports
35/35 released frozen human-designed reference PCBs score 1.0	The tasks are solvable, and the oracle accepts real upstream designs under their declared contracts.
35/35 released structural fail canaries score at most 0.15	The score is not reducible to file existence, KiCad syntax, or component-shell matching.
162 mutation canaries pass validation	The oracle is sensitive to shorts, unrouted nets, isolated pads, missing outlines, and missing high-speed-like routing.
Refdes-renamed USB-C board scores 1.0	The score does not depend on exact connector reference designators.
Plausible USB-C route-width edit remains at least 0.85	The score does not require exact route-shape or width matching when behavior remains plausible.
Tiny-trace USB-C control scores at most 0.35	Manufacturability caps affect scores for physically implausible routing.

Table 5: Complete web-enabled harness baselines. The primary score is difficulty-weighted, and the unweighted score is the mean task score.

Harness	Weighted	Unweighted	Builds
Codex GPT-5.5 web low	0.0124	0.0143	31/35
Codex GPT-5.5 web medium	0.0279	0.0314	35/35
Codex GPT-5.5 web high	0.0293	0.0229	11/35
Codex GPT-5.5 web xhigh	0.0458	0.0500	21/35
Pi Gemini 3.1 Pro Preview web high	0.0113	0.0086	1/35
Pi DeepSeek V4 Pro web high	0.0000	0.0000	0/35

7 Experiments

We evaluate six web-enabled harness configurations on the 35 released tasks. All runs use the same task prompts, task packs, Docker image, and grading protocol. Table 5 reports a May 6, 2026 no-model regrade of saved final projects under the oracle revision used for this manuscript.

No evaluated harness achieves a high absolute score. The main empirical finding is that build success substantially overestimates functional PCB reconstruction. The Codex GPT-5.5 web medium harness builds all 35 released projects but scores only 2.79%. The Codex GPT-5.5 web xhigh harness builds fewer projects than medium but scores higher. These results indicate that executable KiCad output is not a sufficient metric for EDA agents and that reasoning and tool-use choices affect the distribution of functional failures.

Difficulty breakdowns further indicate that the benchmark is not saturated by easy tasks. The Codex GPT-5.5 web xhigh harness reaches 13.75% on medium tasks and 7.08% on hard tasks, but only 2.00% on extreme tasks. The gap grows as boards require larger external contracts, active circuitry, dense routing, or high-speed interfaces.

Statistical interpretation. The result table reports complete released-task sweeps, not repeated independent sampling campaigns. We therefore do not claim statistically significant pairwise model rankings. The claims supported by the table are qualitative and artifact-level: the benchmark is not saturated by evaluated harnesses, build success is not functional success, and access policy materially affects the evaluated harnesses under the same task and grader revision.

7.1 Access-policy calibration

EDA Bench reports access policy as part of the evaluated system. Six no-web Codex calibration sweeps use the same released tasks and grader revision as the web-enabled runs, but disable web access. All six scored zero and produced no buildable projects, indicating that the default prompts and built-in Codex harnesses rely on public-source discovery as well as KiCad operation. These rows are calibration runs rather than closed-book capability measurements for all possible harnesses, so we report them separately from the web-enabled leaderboard.

Table 6: Completed no-web calibration runs. These rows are reported separately from the default web leaderboard because access policy is an evaluated condition.

Harness	Weighted	Unweighted	Builds
Codex GPT-5.5 no-web medium	0.0000	0.0000	0/35
Codex GPT-5.5 no-web high	0.0000	0.0000	0/35
Codex GPT-5.5 no-web xhigh	0.0000	0.0000	0/35
Codex GPT-5.4 no-web xhigh	0.0000	0.0000	0/35
Codex GPT-5.4-mini no-web medium	0.0000	0.0000	0/35
Codex GPT-5.4-mini no-web high	0.0000	0.0000	0/35

8 Failure analysis

The results support three benchmark-level conclusions. First, evaluated frontier-model harnesses are far from solving functional PCB reconstruction: the best evaluated configuration scores only 4.58% overall on the released task set. Second, syntactic validity is not sufficient: a harness can produce buildable KiCad projects for every released task and still fail most functional checks. Third, harness architecture matters: Codex GPT-5.5 configurations with different reasoning settings produce different build and score tradeoffs, so EDA Bench evaluates complete systems rather than base models alone.

Saved grading artifacts expose the main failure modes. Among buildable Codex GPT-5.5 web xhigh submissions, the most common caps were split required external-net continuity, isolated external-I/O pads, wrong external-I/O pin/net mapping, missing or weak component-function realization, missing or implausible high-speed pair geometry, and weak active-device power integrity. The all-building Codex GPT-5.5 web medium run shows a similar pattern: wrong external mapping, missing component-function realization, split required external nets, same-layer trace shorts, and isolated external-I/O pads.

These caps correspond to concrete functional failures rather than parser failures. Submissions often instantiate connector footprints with plausible labels, but leave required I/O pads isolated, split required nets across disconnected copper, swap external pin semantics, omit active circuitry, or route high-speed-like interfaces implausibly. These errors explain why KiCad parseability and component presence are weak proxies for functional PCB reconstruction.

9 Related work

Execution-based software-agent benchmarks. Execution-based software benchmarks evaluate agents by running the artifacts they produce. SWE-bench frames software engineering as repository-level issue resolution, where an agent edits a real codebase and must pass tests associated with the target issue [Jimenez et al., 2023]. OSWorld evaluates multimodal agents in real desktop and web environments with custom execution-based evaluation scripts [Xie et al., 2024]. ProgramBench studies a more reconstructive setting in which agents rebuild software from an executable and documentation, and behavioral tests are generated to compare the candidate program with the reference executable [Yang et al., 2026]. EDA Bench adopts this behavioral-evaluation perspective but applies it to PCB reconstruction, where correctness depends on realized geometry, external connectivity, and electrical-interface behavior rather than software tests alone.

CAD generation and geometric evaluation. Text2CAD introduced a large-scale text-to-parametric-CAD dataset and model for generating CAD command sequences from natural-language prompts [Khan et al., 2024]. CadEval and CAD Arena provide evaluation settings for text-to-CAD models using geometric validity, rendering, and prompt-level comparisons [Epoch AI, 2026, CAD Arena, 2026]. CADSmith further emphasizes programmatic geometric validation for generated CadQuery models [Barkley et al., 2026]. These works motivate execution-based evaluation for engineering artifacts, but their primary correctness signals are geometric rather than electrical. PCB reconstruction requires connectivity, routing, short avoidance, active-circuit realization, and external-I/O behavior in addition to shape or layout plausibility.

Circuit, schematic, and PCB benchmarks. Recent EDA benchmarks study several adjacent tasks. CircuitLM and PCBSchemaGen generate or synthesize circuit schematics from natural-language prompts and use structural or constraint-based checks to improve schematic validity [Al Hasan et al., 2026, Zou et al., 2026]. SchGen studies PCB schematic generation using semantic-grounded code representations [Luo et al., 2026]. OmniSch evaluates multimodal schematic understanding and schematic-to-graph reasoning over real-world diagrams [Lu et al., 2026]. HWE-Bench evaluates board-level schematic design with datasheet context and hardware bug repair over repository-scale hardware projects [Qiu et al., 2026, Cui et al., 2026]. PCB-Bench evaluates LLM reasoning over placement, routing, and PCB design comprehension using text, images, and complete PCB projects [Li et al., 2026]. EDA Bench differs by requiring an agent to produce a complete routed KiCad PCB project and by scoring the realized board through external-I/O simulation and PCB-geometry checks rather than schematic-only validity, visual reasoning, or placement/routing question answering.

10 Limitations

EDA Bench is not a hidden-answer or closed-book benchmark. The released tasks are public-source, and upstream designs may be discovered through web search. This determines the supported claim: scores measure EDA task solving under fixed prompts, tools, task-pack revisions, provenance, and access policy, not closed-book invention.

The task set is intentionally small compared with text benchmarks. It has 35 released tasks because each task requires a real source design, a reference project, task contracts, canaries, license review, and grader support. Breadth should grow through additional source-backed packs or a sealed private split, not by adding weakly checked or weakly specified tasks.

The oracle is functional but not a substitute for manufacturing review. It uses deterministic generic behavioral models. It does not prove electrical safety, regulatory compliance, high-speed protocol compliance, thermal reliability, power sequencing correctness, or production readiness. Richer per-task SPICE, IBIS, S-parameter, and field-solver models would improve fidelity. The calibrated task-family checks are deliberately conservative: they tighten measured oracle signals for four representative board families, but they are not vendor-accurate silicon or channel simulations.

Task artifacts inherit mixed upstream hardware licenses. The public artifact includes only score-supported tasks whose runtime and grading packs can be redistributed under the release policy.

11 Conclusion

EDA Bench evaluates whether language-model agents can reconstruct PCB designs that function at their electrical interfaces. It is built from real public hardware projects, scored by explicit I/O simulation, and evaluated as a full harness rather than a single text response. Initial results show that evaluated systems often produce syntactically valid KiCad artifacts, but rarely produce boards that satisfy functional boundary contracts. This discrepancy is the central evaluation target of EDA Bench.

References

- Khandakar Shakib Al Hasan, Syed Rifat Raiyan, Hasin Mahtab Alvee, and Wahid Sadik. CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts. arXiv:2601.04505, 2026. URL: <https://arxiv.org/abs/2601.04505>.
- Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. CADSmith: Multi-Agent CAD Generation with Programmatic Geometric Validation. arXiv:2603.26512, 2026. URL: <https://arxiv.org/abs/2603.26512>.
- CAD Arena. CAD Arena: Open Benchmark for AI-Generated Parametric CAD. 2026. URL: <https://cadarena.dev/>.
- Fan Cui, Hongyuan Hou, Zizhang Luo, Chenyun Yin, and Yun Liang. HWE-Bench: Benchmarking LLM Agents on Real-World Hardware Bug Repair Tasks. arXiv:2604.14709, 2026. URL: <https://arxiv.org/abs/2604.14709>.

- Epoch AI. CadEval. 2026. URL: <https://epoch.ai/benchmarks/cad-eval>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. URL: <https://arxiv.org/abs/2310.06770>.
- Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL: <https://arxiv.org/abs/2409.17106>.
- KiCad Project. KiCad Documentation. 2026. URL: <https://docs.kicad.org/>.
- Jindong Li, Lianrong Chen, Bin Yang, Jiadong Zhu, Ying Wang, Yuzhe Ma, and Menglin Yang. PCB-Bench: Benchmarking LLMs for Printed Circuit Board Placement and Routing. In *International Conference on Learning Representations*, 2026. URL: <https://openreview.net/forum?id=Q5QLu7XTWx>.
- Taiting Lu, Kaiyuan Lin, Yuxin Tian, Yubo Wang, Muchuan Wang, Sharique Khatri, Akshit Kartik, Yixi Wang, Amey Santosh Rane, Yida Wang, Yifan Yang, Yi-Chao Chen, Yincheng Jin, and Mahanth Gowda. OmniSch: A Multimodal PCB Schematic Benchmark For Structured Diagram Visual Reasoning. arXiv:2604.00270, 2026. URL: <https://arxiv.org/abs/2604.00270>.
- Qinpei Luo, Ruichun Ma, Xinyu Zhang, and Lili Qiu. SchGen: PCB Schematic Generation with Semantic-Grounded Code Representations. OpenReview preprint, 2026. URL: <https://openreview.net/forum?id=TyWs6rWWHb>.
- ngspice contributors. ngspice, the open-source SPICE circuit simulator. 2026. URL: <https://ngspice.sourceforge.io/>.
- Weibo Qiu, Yinhao Xiao, and Runyu Pan. HWE-Bench: Can Language Models Perform Board-level Schematic Designs? arXiv:2603.18102, 2026. URL: <https://arxiv.org/abs/2603.18102>.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. URL: <https://arxiv.org/abs/2404.07972>.
- John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. ProgramBench: Can Language Models Rebuild Programs From Scratch? arXiv:2605.03546, 2026. URL: <https://arxiv.org/abs/2605.03546>.
- Huanghaohe Zou, Peng Han, Emad Nazerian, and Alex Q. Huang. PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards. arXiv:2602.00510, 2026. URL: <https://arxiv.org/abs/2602.00510>.

A Artifact availability and provenance

The source repository contains the runner and harness code, task-pack loader and grader support, public documentation, package tests, and the task-pack manifest. Python wheels and source distributions exclude concrete task instances, reference projects, upstream snapshots, and canary submissions. The Hugging Face task dataset contains runtime task packs, full grader task packs, prompts, references, upstream snapshots, canary submissions, task metadata, Croissant metadata, and license/provenance notes.

The public task dataset is hosted at <https://huggingface.co/datasets/eda-bench-neurips-2026/eda-bench-tasks>. The results use task-pack revision c64b9169d6d8c0d73614e14c7050cf036aeb1559. The public provenance dataset is hosted at <https://huggingface.co/datasets/eda-bench-neurips-2026/eda-bench-provenance>. The public code release is hosted at <https://github.com/CAD-bench/eda-bench>.

Run provenance includes task prompts, command lines, usage summaries, grader outputs, final projects, and generated artifacts. Storage-light campaigns can omit full Docker image uploads and raw model transcripts, but retain final reports, final projects, grading artifacts, and byte-count summaries for omitted transcript content.

B Validation controls

Table 7: Validation controls for score interpretation.

Control	Observed result
Frozen reference project	35/35 released references score 1.0
Structural fail canary	35/35 released fail canaries score at most 0.15, below the 0.75 release threshold
Generated mutation canaries	162 generated mutation canaries across released task families pass validation
Calibrated task-family checks	USB-C, RS-485, BQ24295, and M.2/PCIe references score 1.0 and their mutation canaries remain below threshold
Refdes-invariant equivalent board	USB-C reference with renamed connector refdes scores 1.0
Missing routed copper	USB-C and NanoUPDI route-removal controls score 0.0
Plausible route-width edit	USB-C route-width edit remains at least 0.85
Implausibly tiny traces	Manufacturability cap applies and USB-C tiny-trace control scores at most 0.35
Missing or unparsable project	Returns 0.0 and build failure

C Failure-mode diagnostics

Table 8: Common functional failure modes exposed by the oracle. These failures are typical of buildable submissions and illustrate why KiCad parseability is not sufficient for functional PCB reconstruction.

Failure mode	Observable symptom	Why it affects score
Split required external nets	Pins that should belong to the same external interface are assigned to disconnected copper regions or separate nets	Boundary stimuli cannot propagate through the declared interface, even when connector footprints are present
Isolated external I/O pads	Required pads are placed but not connected to routed copper or active circuitry	The board exposes a physical pad without realizing the electrical behavior required by the contract
Wrong pin or net mapping	External connector pins are swapped, mislabeled, or attached to semantically incorrect nets	The board may appear visually plausible while driving or observing the wrong external signal
Missing active realization	Components required for level shifting, power-path behavior, transceiver behavior, or control logic are absent or not connected	Passive connectivity alone cannot implement the task-required circuit behavior
Weak active-device power integrity	Active devices are present but lack required power, ground, or enable connectivity	The simulated or inferred behavior cannot be attributed to a realizable powered circuit
Implausible high-speed geometry	Differential or high-speed-like interface families are missing, weakly routed, or routed with implausible geometry	Dense carrier and compute tasks require not only pin labels but routed interface families with plausible physical realization

D Baseline provenance

The May 6, 2026 result table regrades saved final projects without rerunning model harnesses. Source runs were: `codex__gpt-5.5-web-low_20260505T200208Z_2719115`, `codex__gpt-5.5-web-medium_20260506T010715Z_2812175`, `codex__gpt-5.5-web-medium_20260506T010715Z_2812182`, `codex__gpt-5.5-web-medium_20260506T032737Z_2884928`, `codex__gpt-5.5-web-medium_20260506T010716Z_2812190`, `codex__gpt-5.5-web-high_20260505T200208Z_2719113`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884940`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884938`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884943`, `codex__gpt-5.5-web-xhigh_20260506T034126Z_2891808`, `pi__gemini-3.1-pro-preview-web-high_20260505T184850Z_2697190`, and `pi__deepseek__deepseek-v4-pro-web-high_20260505T200208Z_2719116`.

The executable regrade entry point is `uv run regrade-provenance`. For example, the Codex GPT-5.5 web medium row is reproduced by passing its four source runs:

```
uv run regrade-provenance \  
  --run codex__gpt-5.5-web-medium_20260506T010715Z_2812175 \  
  --run codex__gpt-5.5-web-medium_20260506T010715Z_2812182 \  
  --run codex__gpt-5.5-web-medium_20260506T032737Z_2884928 \  
  --run codex__gpt-5.5-web-medium_20260506T010716Z_2812190
```

By default the command uses the released task manifest, downloads saved `submission/final_project` directories from Hugging Face provenance, and runs the local oracle.

Completed no-web calibration runs are present under these provenance paths: `evals/harnesses/codex__gpt-5.5-no_web-medium_20260507T033319Z_3432012`, `evals/harnesses/codex__gpt-5.5-no_web-high_20260507T033319Z_3432018`, `evals/harnesses/codex__gpt-5.5-no_web-xhigh_20260507T033319Z_3432017`, `evals/harnesses/codex__gpt-5.4-no_web-xhigh_20260507T033528Z_3439753`, `evals/harnesses/codex__gpt-5.4-mini-no_web-medium_20260507T033724Z_3448333`, and `evals/harnesses/codex__gpt-5.4-mini-no_web-high_20260507T033724Z_3448341`.

E Cost and compute accounting

The completed GPT-5.5 web runs used storage-light provenance. The combined Codex GPT-5.5 low/medium/high/xhigh campaign uploaded about 185.6 MiB of Hugging Face provenance. Across all six final harnesses, the total uploaded provenance was about 193.1 MiB.

Table 9: Recorded Codex GPT-5.5 web campaign accounting.

Harness	Estimated OpenAI API USD	Estimated Codex credits	HF provenance
Codex GPT-5.5 web low	38.22	955.41	48.9 MB
Codex GPT-5.5 web medium	84.40	2110.05	49.5 MB
Codex GPT-5.5 web high	49.39	1234.64	27.6 MB
Codex GPT-5.5 web xhigh	182.75	4568.64	68.6 MB

The estimates are computed from recorded token usage, current provider pricing, and the Codex token-based rate card. They are included to make leaderboard submissions auditable and to discourage hidden cost and provenance blowups.

F License and redistribution

EDA Bench separates runtime prompts from full task packs with references, snapshots, canaries, and grader contracts. Released task packs include source-backed materials with redistributable prompts, references, grader contracts, canaries, and license/provenance notes. This keeps the benchmark artifact reproducible without mixing task instances whose upstream terms are incompatible with public redistribution.